

Edukia — API Fichiers (/files/...) pour le mobile

Guide d'intégration du système de fichiers (upload & download via Cloudflare R2, URLs présignées). Exemples issus d'**appels réels** (signatures/hôte rédigés).

1. Principe

Le backend ne transporte **jamais** les octets : il émet des **URLs présignées** courtes ; le client lit/écrit **directement** sur R2.

Deux familles de **slots** (voir `GET /files/registry`) :

- **Public** (ex. `utilisateurs.profil`) — lisible anonymement. L'URL publique

complète est stockée sur la ligne (`<slot>_path`) ; aucun appel de download.

- **Privé** (ex. `epreuves.file`, `prestataires.identity`) — lecture uniquement

via une URL présignée temporaire (`download-url`).

Auth. Tous les endpoints `/files/...` exigent `Authorization: Bearer <token>`.

2. Découverte — `GET /files/registry`

Renvoie, par entité/slot, les extensions autorisées et la visibilité.

```
{
  "utilisateurs": { "profil": { "authorized": ["jpg","jpeg","png","webp","avif"], "public": true } },
  "epreuves":     { "file":    { "authorized": ["pdf"], "public": false } }
}
```

3. Upload (2 étapes) — public ou privé

Étape 1 — demander l'URL présignée

`POST /files/:entity/:uuid/:slot/upload-url` — corps : `{ "extension": "png" }`.

```
// PUBLIC (utilisateurs.profil)
{
  "url": "https://<compte>.r2.cloudflarestorage.com/utilisateurs/0f8b.../profil.png?X-Amz-Algorithm=...&X-Amz-Credential=
<REDACTED>&X-Amz-Expires=600&X-Amz-Signature=<REDACTED>...",
  "method": "PUT",
  "content_type": "image/png",
  "required_headers": {
    "Content-Type": "image/png",
    "Cache-Control": "public, max-age=31536000, immutable"
  },
  "path": "https://assets-dev.edukia.net/utilisateurs/0f8b.../profil.png?v=1783494610074",
  "extension": "png",
  "expires_in": 600,
  "public": true
}
```

Étape 2 — envoyer les octets à R2

`PUT <url>` avec le **fichier brut** en corps. **Rejouez** `required_headers` à l'identique :

- `Content-Type` : **toujours**.
- `Cache-Control` : **pour les slots publics** — sinon R2 renvoie

`401 SignatureDoesNotMatch` (la signature couvre ce header).

```
curl -X PUT "<url>" \
-H "Content-Type: image/png" \
-H "Cache-Control: public, max-age=31536000, immutable" \
--data-binary @photo.png
# => 200
```

Le backend a déjà écrit `<slot>_path` (et l'extension) sur la ligne à l'étape 1.

- **TTL de l'URL d'upload** : 10 min (`expires_in: 600`) par défaut ; certains slots 1 h (`epreuves.file` / `concours.file` = 3600).

4. ⚠ Re-upload & cache (public) — important

La clé R2 est **déterministe** (`<entity>/<uuid>/<slot>.<ext>`) et les objets publics sont servis `immutable` (cache 1 an). Un ré-upload écrase bien l'objet, mais l'URL ne changeait pas → les CDN/navigateurs servaient l'**ancienne** copie.

Corrigé : `<slot>_path` porte désormais un **jeton de version** `?v=<ts>` qui **change à chaque upload**. Conséquence pour le mobile :

Ne mémorisez pas l'URL en dur. Après un upload, **relisez l'entité** (`GET /utilisateurs/profil` , etc.) et utilisez la **nouvelle** valeur de `<slot>_path` (nouveau `?v=`) — l'image fraîche s'affiche immédiatement.

5. Download

Slot PUBLIC — lecture directe (pas d'appel `download-url`)

L'URL complète est déjà sur la ligne : lisez `utilisateur.profil` (= `<slot>_path`) et faites un **GET simple** dessus.

```
GET https://assets-dev.edukia.net/utilisateurs/0f8b.../profil.png?v=1783494610074 → 200 (image)
```

Slot PRIVÉ — URL présignée temporaire

`GET /files/:entity/:uuid/:slot/download-url` (400 pour un slot public).

```
{
  "url": "https://<compte>.r2.cloudflarestorage.com/epreuves/6ce5.../file.pdf?...&X-Amz-Signature=<REDACTED>&...",
  "method": "GET",
  "extension": "pdf",
  "expires_in": 3600,
  "public": false
}
```

Faites un **GET** sur `url` pour récupérer le fichier. Le lien expire (`expires_in`) — 10 min par défaut, **1 h** pour `epreuves.file` / `concours.file` (gros PDF). `404` si aucun fichier n'a encore été enregistré.

6. Règles & pièges

Cas	Comportement
Corps de <code>upload-url</code>	<code>{ "extension": "<ext>" }</code> uniquement — pas de <code>content_type</code> .
Extension refusée	<code>400</code> (voir <code>authorized</code> dans le registre).
PUT sans <code>Cache-Control</code> sur slot public	<code>401 SignatureDoesNotMatch</code> .
<code>download-url</code> sur slot public	<code>400</code> — lisez plutôt <code><slot>_path</code> .
Après upload	relire l'entité pour obtenir le <code><slot>_path</code> à jour (<code>?v=</code>).
URL expirée	redemandez <code>upload-url</code> / <code>download-url</code> .

7. Exemple Flutter / Dart (upload d'une photo de profil, slot public)

```
final base = 'https://api.edukia.net';
final auth = { 'Authorization': 'Bearer $jwt' };

// 1) URL présignée
final pres = jsonDecode(await http.post(
  Uri.parse('$base/files/utilisateurs/$uuid/profil/upload-url'),
  headers: { ...auth, 'Content-Type': 'application/json' },
  body: jsonEncode({ 'extension': 'png' }))).body);

// 2) PUT direct sur R2 en jouant required_headers
await http.put(Uri.parse(pres['url']),
  headers: Map<String, String>.from(pres['required_headers']),
  body: bytes); // ⇒ 200

// 3) relire le profil → nouvelle URL (?v=) prête à afficher
final profil = jsonDecode(await http.get(
  Uri.parse('$base/utilisateurs/profil'), headers: auth)).body);
final photoUrl = profil['profil']; // https://assets-dev.edukia.net/.../profil.png?v=...
```

8. Exemple bout-en-bout (curl)

```
BASE=https://api.edukia.net
# --- PUBLIC : upload photo de profil ---
PRES=$(curl -s -X POST "$BASE/files/utilisateurs/$UUID/profil/upload-url" \
  -H "Authorization: Bearer $JWT" -H "Content-Type: application/json" \
  -d '{"extension":"png"}')
URL=$(echo "$PRES" | jq -r .url)
curl -X PUT "$URL" -H "Content-Type: image/png" \
  -H "Cache-Control: public, max-age=31536000, immutable" --data-binary @photo.png # ⇒ 200
# lecture : GET simple sur le champ profil (avec ?v=) renvoyé par /utilisateurs/profil

# --- PRIVÉ : download d'une épreuve ---
DL=$(curl -s "$BASE/files/epreuves/$UUID/file/download-url" -H "Authorization: Bearer $JWT")
curl -L "$(echo "$DL" | jq -r .url)" -o epreuve.pdf # ⇒ 200 (lien valable expires_in s)
```